SQL Project 1

Scenario: City Metro Transit System

1. Scenario Overview

A city metro rail operator runs multiple train lines connecting a network of stations across town. Passengers register with the metro service and book tickets to travel from one station to another on a specific train. This database models that entire operation: the stations in the network, the trains that run on different route lines, the registered passengers, and every ticket booking made.

The dataset contains four connected tables and approximately 2,000 rows in total:

- Stations — 25 rows
- Trains — 20 rows
- Passengers — 465 rows
- TicketBookings — 1,490 rows

Total: 2,000 rows combined across all four tables.

2. How the Tables Connect

TicketBookings is the central table and links to the other three:

- Each booking belongs to one Passenger (passenger_id → Passengers)
- Each booking is made on one Train (train_id → Trains)
- Each booking has a boarding Station (boarding_station_id → Stations)
- Each booking has a destination Station (destination_station_id → Stations)

Note that TicketBookings references the Stations table twice (once for boarding, once for destination) — a good example for students of a table having more than one foreign key pointing to the same parent table.

3. Table 1: Stations

Master list of metro stations across the city.

```
CREATE TABLE Stations (
    station_id INT PRIMARY KEY,
    station_name VARCHAR(50) NOT NULL UNIQUE,
    city VARCHAR(50) NOT NULL,
    zone VARCHAR(20),
    opened_year INT CHECK (opened_year >= 1990)
);
```

Sample data (full 25 rows shown — small enough to include completely):

```
INSERT INTO Stations (station_id, station_name, city, zone, opened_year) VALUES
(1, 'Central Junction', 'Bengaluru', 'North', 2021),
(2, 'Riverside', 'Hyderabad', 'South', 2005),
(3, 'Old Fort', 'Chennai', 'North', 2019),
(4, 'Tech Park', 'Kochi', 'North', 2016),
(5, 'Lakeview', 'Pune', 'North', 1998),
(6, 'Market Square', 'Bengaluru', 'South', 2005),
(7, 'University Gate', 'Kochi', 'Central', 1998),
(8, 'Hillside', 'Kochi', 'South', 2020),
(9, 'Airport Link', 'Kochi', 'West', 2005),
(10, 'Harbor Point', 'Pune', 'Central', 2006),
(11, 'Greenfield', 'Bengaluru', 'South', 2020),
(12, 'North Terminus', 'Pune', 'East', 2006),
(13, 'South Terminus', 'Chennai', 'South', 2022),
(14, 'Museum Road', 'Hyderabad', 'North', 2000),
(15, 'Stadium', 'Pune', 'North', 2009),
(16, 'Industrial Area', 'Hyderabad', 'Central', 2006),
(17, 'City Center', 'Bengaluru', 'West', 2015),
(18, 'East Gate', 'Bengaluru', 'West', 2000),
(19, 'West Gate', 'Kochi', 'East', 2018),
(20, 'Botanical Garden', 'Kochi', 'East', 2016),
(21, 'Heritage Circle', 'Chennai', 'North', 1999),
(22, 'Metro Mall', 'Chennai', 'East', 2000),
(23, 'Sunrise Colony', 'Chennai', 'North', 2010),
(24, 'Palace Road', 'Hyderabad', 'West', 2018),
(25, 'Convention Center', 'Hyderabad', 'South', 2009);
```

4. Table 2: Trains

Trains operating on different route lines.

```
CREATE TABLE Trains (
    train_id INT PRIMARY KEY,
    train_name VARCHAR(50) NOT NULL UNIQUE,
    route_line VARCHAR(20) NOT NULL,
    capacity INT CHECK (capacity > 0)
);
```

Sample data (full 20 rows shown — small enough to include completely):

```
INSERT INTO Trains (train_id, train_name, route_line, capacity) VALUES
(1, 'Blue Line Express', 'Line 3', 220),
(2, 'Blue Line Local', 'Line 3', 180),
(3, 'Green Line Express', 'Line 5', 220),
(4, 'Green Line Local', 'Line 5', 220),
(5, 'Red Line Express', 'Line 2', 300),
(6, 'Red Line Local', 'Line 4', 250),
(7, 'Yellow Line Express', 'Line 5', 220),
```

```
(8, 'Yellow Line Local', 'Line 3', 180),
(9, 'Purple Line Express', 'Line 2', 180),
(10, 'Purple Line Local', 'Line 3', 300),
(11, 'Orange Line Express', 'Line 3', 180),
(12, 'Orange Line Local', 'Line 2', 320),
(13, 'Silver Line Express', 'Line 3', 220),
(14, 'Silver Line Local', 'Line 4', 300),
(15, 'Violet Line Express', 'Line 4', 220),
(16, 'Violet Line Local', 'Line 3', 220),
(17, 'Pink Line Express', 'Line 2', 320),
(18, 'Pink Line Local', 'Line 5', 250),
(19, 'Teal Line Express', 'Line 5', 300),
(20, 'Teal Line Local', 'Line 5', 300);
```

5. Table 3: Passengers

Registered metro passengers.

```
CREATE TABLE Passengers (
    passenger_id INT PRIMARY KEY,
    passenger_name VARCHAR(50) NOT NULL,
    age INT CHECK (age >= 5),
    email VARCHAR(100) UNIQUE,
    city VARCHAR(50)
);
```

Sample data (first 15 of 465 rows — full data in the accompanying metro_transit_full.sql file):

```
INSERT INTO Passengers (passenger_id, passenger_name, age, email, city) VALUES
(1, 'Nisha Singh', 25, 'nisha.singh1@metromail.com', 'Coimbatore'),
(2, 'Divya Sharma', 22, 'divya.sharma2@metromail.com', 'Hyderabad'),
(3, 'Arjun Das', 16, 'arjun.das3@metromail.com', 'Delhi'),
(4, 'Ashok Kapoor', 67, 'ashok.kapoor4@metromail.com', 'Kochi'),
(5, 'Sunita Kumar', 22, 'sunita.kumar5@metromail.com', 'Kochi'),
(6, 'Swati Iyer', 45, 'swati.iyer6@metromail.com', 'Delhi'),
(7, 'Arjun Choudhary', 8, 'arjun.choudhary7@metromail.com', 'Kochi'),
(8, 'Mahesh Nair', 72, 'mahesh.nair8@metromail.com', 'Chennai'),
(9, 'Ritu Bose', 33, 'ritu.bose9@metromail.com', 'Hyderabad'),
(10, 'Nisha Nair', 75, NULL, 'Bengaluru'),
(11, 'Vijay Reddy', 70, 'vijay.reddy11@metromail.com', 'Bengaluru'),
(12, 'Anjali Pillai', 47, 'anjali.pillai12@metromail.com', 'Pune'),
(13, 'Sneha Singh', 18, 'sneha.singh13@metromail.com', 'Chennai'),
(14, 'Preeti Gupta', 24, 'preeti.gupta14@metromail.com', 'Hyderabad'),
(15, 'Arun Nayar', 29, 'arun.nayar15@metromail.com', 'Kochi'),
... -- 465 total passenger rows in the accompanying .sql file
```

6. Table 4: TicketBookings

Every ticket booking made by a passenger.

```
CREATE TABLE TicketBookings (
    booking_id INT PRIMARY KEY,
    passenger_id INT,
    train_id INT,
    boarding_station_id INT,
    destination_station_id INT,
    booking_date DATE,
    fare DECIMAL(10,2) CHECK (fare > 0),
    seat_class VARCHAR(20) DEFAULT 'General',
    travel_time_minutes INT CHECK (travel_time_minutes > 0),
    FOREIGN KEY (passenger_id) REFERENCES Passengers(passenger_id),
    FOREIGN KEY (train_id) REFERENCES Trains(train_id),
    FOREIGN KEY (boarding_station_id) REFERENCES Stations(station_id),
    FOREIGN KEY (destination_station_id) REFERENCES Stations(station_id)
);
```

Sample data (first 15 of 1,490 rows — full data in the accompanying metro_transit_full.sql file):

```
INSERT INTO TicketBookings (booking_id, passenger_id, train_id, boarding_station_id,
destination_station_id, booking_date, fare, seat_class, travel_time_minutes) VALUES
(1, 377, 2, 23, 5, '2026-06-06', 107.55, 'Premium', 34),
(2, 245, 17, 13, 11, '2026-04-23', 165.5, 'General', 66),
(3, 182, 9, 20, 16, '2026-04-25', 61.04, 'General', 39),
(4, 153, 10, 23, 7, '2026-06-28', 67.31, 'General', 70),
(5, 408, 9, 10, 4, '2026-06-13', 104.63, 'Premium', 94),
(6, 203, 12, 25, 5, '2026-05-08', 62.46, 'General', 13),
```

```
(7, 178, 15, 21, 9, '2026-07-05', 50.31, 'Premium', 69),
(8, 139, 18, 23, 9, '2026-04-18', 116.59, 'General', 21),
(9, 125, 2, 22, 17, '2026-04-29', 53.42, 'General', 89),
(10, 170, 16, 4, 22, '2026-07-08', 15.85, 'General', 25),
(11, 81, 14, 21, 16, '2026-06-01', 47.91, 'Premium', 91),
(12, 165, 10, 21, 2, '2026-07-08', 122.63, 'General', 19),
(13, 379, 2, 6, 14, '2026-07-17', 169.1, 'General', 30),
(14, 429, 13, 16, 6, '2026-07-05', 159.66, 'General', 45),
(15, 291, 20, 4, 11, '2026-05-07', 178.2, 'General', 66),
... -- 1,490 total booking rows in the accompanying .sql file
```

7

The 6 sections below mirror the topics covered across all the earlier practice files. Each question includes one short hint naming the concept to apply — work out the query yourself using that hint.

Section A: Constraints & Table Design

Q1. A new booking is inserted with a destination_station_id that does not exist in the Stations table. What happens, and why?
Hint: *FOREIGN KEY constraint*

Q2. Try inserting a passenger with a duplicate email address. What error do you expect?
Hint: *UNIQUE constraint*

Q3. Insert a booking without mentioning seat_class. What value does it get automatically?
Hint: *DEFAULT constraint*

Q4. Insert a passenger with age = 2. What happens?
Hint: *CHECK constraint*

Section B: Aggregate Functions & Text Functions

Q5. Find the total number of bookings made in 'Premium' seat class.
Hint: *COUNT() with WHERE*

Q6. Find the average fare paid across all bookings.
Hint: *AVG()*

Q7. Find the total number of distinct cities passengers come from.
Hint: *COUNT(DISTINCT ...)*

Q8. Display each station's name in uppercase along with its city.
Hint: *UPPER()*

Q9. Find the length of each passenger's name.
Hint: *LENGTH()*

Section C: Date & Time Functions

Q10. Find the total number of bookings made in the month of June 2026.
Hint: *MONTH() and YEAR()*

Q11. Find the day name (Monday, Tuesday, etc.) for each booking_date.
Hint: *DAYNAME()*

Q12. Find how many days ago each booking was made, counting from today.
Hint: *DATEDIFF()*

Q13. Add 30 days to each booking_date to find a hypothetical renewal date.
Hint: *DATE_ADD()*

Section D: Joins + GROUP BY + HAVING

Q14. List every booking along with the passenger's name and the train name used.
Hint: *JOIN across TicketBookings, Passengers, and Trains*

Q15. Find the total number of bookings made for each train.
Hint: *JOIN + GROUP BY*

Q16. Find trains that have been booked more than 60 times.
Hint: *GROUP BY + HAVING*

Q17. Find the total fare collected at each boarding station.
Hint: *JOIN Stations on boarding_station_id + GROUP BY*

Q18. Find stations that have never been used as a boarding station.
Hint: *LEFT JOIN with WHERE ... IS NULL*

Section E: Subqueries, CTEs & Views

Q19. Find all bookings made by the passenger named 'Nisha Singh'.
Hint: *Subquery in WHERE*

Q20. Find all bookings made on trains belonging to 'Line 3' or 'Line 5'.
Hint: *Subquery with IN*

Q21. Using a CTE, find the total fare collected per passenger.
Hint: *Basic CTE*

Q22. Using a CTE joined with Passengers, show passenger name alongside their total number of bookings.
Hint: *CTE with JOIN*

Q23. Create a view that shows booking_id, passenger name, train name, and fare for every booking.
Hint: *CREATE VIEW with JOIN*

Section F: Window Functions, CASE & Transactions

Q24. Rank all bookings by fare in descending order.
Hint: *RANK() OVER (ORDER BY ...)*

Q25. Rank bookings by fare separately within each train.
Hint: *PARTITION BY*

Q26. For each passenger's bookings ordered by booking_date, show the previous booking's fare.
Hint: *LAG()*

Q27. Label each booking as 'High Fare' (above 100), 'Medium Fare' (50-100), or 'Low Fare' (below 50).
Hint: *CASE statement*

Q28. Start a transaction, delete a booking, confirm it's gone, then undo the deletion.
Hint: *ROLLBACK*

Q29. Create an index to speed up searches that filter bookings by passenger_id.
Hint: *CREATE INDEX*

Q30. Rank each passenger's bookings by fare, giving the same rank to equal fares without leaving gaps in the ranking sequence.
Hint: *DENSE_RANK()*

Q31. For each boarding station, rank the bookings made from it by fare, highest first.
Hint: *PARTITION BY (window function)*

Q32. For each passenger's bookings ordered by booking_date, show the next booking's fare next to the current one.
Hint: *LEAD()*

Q33. For each train, calculate a running total of fare collected, ordered by booking_date, using only the current and previous row.
Hint: *Window frame - ROWS BETWEEN 1 PRECEDING AND CURRENT ROW*

Q34. Create a composite index on boarding_station_id and destination_station_id together, then view all indexes on the TicketBookings table.
Hint: *Composite index + SHOW INDEX*